LAPORAN

WORKSHOP Mobile Web Server

(TIF310808)

Acara 10

SEMESTER



Nama Tugas

NHERITANCE, BASE CONTROLLER, BASE MODEL, REPOSITORY PATTERN

Nama:

Adam Ridho Saifulloh

NIM:

E41250216

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Pengembangan aplikasi web backend membutuhkan struktur program yang terorganisir agar mudah dipahami, diuji, dipelihara, dan dikembangkan. Pada Acara 10 dipelajari penerapan konsep Object-Oriented Programming (OOP), khususnya inheritance dan Repository Pattern. Konsep tersebut diterapkan pada aplikasi Manajemen Data Mahasiswa.

Pada project yang dikerjakan, fungsi umum controller ditempatkan pada BaseController, sedangkan kebutuhan akses database dasar ditempatkan pada BaseModel. MahasiswaController mewarisi BaseController, sementara MahasiswaRepository mewarisi BaseModel dan menangani query database. Dengan pembagian tersebut, Controller tidak perlu menuliskan query SQL secara langsung.

1.2 Tujuan

- Memahami konsep inheritance pada pemrograman berorientasi objek.
- Menerapkan BaseController sebagai class induk controller.
- Menerapkan BaseModel sebagai class dasar untuk akses database.
- Menerapkan Repository Pattern untuk memisahkan akses database dari logika controller.
- Menguji fitur tampil, tambah, ubah, dan hapus data mahasiswa.

1.3 Pokok Bahasan

Inheritance, Base Controller, Base Model, Repository Pattern.

Sesuai BKPM, Acara 10 dilaksanakan pada Minggu 5/Pertemuan 1 dengan alokasi waktu 2 × 170 menit. □filecite□turn6file0□

BAB II DASAR TEORI

2.1 Inheritance

Inheritance atau pewarisan adalah konsep OOP yang memungkinkan sebuah class mewarisi atribut dan method dari class lain. Pada aplikasi MVC, fungsi umum seperti view() dan redirect() dapat ditempatkan pada BaseController sehingga controller lain cukup mewarisinya.

2.2 Base Controller

BaseController merupakan class induk controller. Pada project, class ini menyediakan method protected view() untuk memuat View dan redirect() untuk mengarahkan halaman sekaligus menyimpan session alert.

2.3 Base Model

BaseModel menjadi class induk untuk class yang membutuhkan koneksi database. Class ini menerima objek Database melalui constructor dan menyimpan koneksi PDO pada property protected $pdo.

2.4 Repository Pattern

Repository Pattern memusatkan proses akses data dan query SQL pada Repository. Controller hanya memanggil method seperti all(), find(), create(), update(), dan delete(), tanpa menulis SQL secara langsung.

2.5 Hubungan Antarclass

- BaseController → diwarisi MahasiswaController.
- BaseModel → diwarisi MahasiswaRepository.
- Database → menyediakan koneksi PDO kepada BaseModel melalui constructor.
- MahasiswaRepository → mengelola query data mahasiswa.
- MahasiswaController → mengatur alur request dan memanggil Repository.

BAB III ALAT DAN BAHAN

- Laptop/PC
- Web Browser
- Visual Studio Code
- XAMPP/Laragon
- PHP
- HTML, CSS, Bootstrap
- Modul Praktikum

Daftar alat dan bahan mengikuti bagian Alat dan Bahan pada BKPM Acara 10.
□filecite□turn6file0□

BAB IV PROSEDUR KERJA

Prosedur kerja berikut disusun mengikuti prosedur Acara 10 pada BKPM dan disesuaikan dengan struktur source code yang terdapat pada ZIP project.

1. Membuat BaseController yang berisi method view() dan redirect().

2. Membuat MahasiswaController yang mewarisi (extends) BaseController dan menggunakan method yang diwariskan untuk menampilkan halaman data mahasiswa.

3. Membuat MahasiswaRepository yang menangani operasi database, yaitu menampilkan, menambah, mengubah, dan menghapus data mahasiswa.

4. Memodifikasi MahasiswaController agar seluruh proses akses data dilakukan melalui MahasiswaRepository, tanpa menuliskan query SQL di dalam Controller.

5. Menguji aplikasi dengan menjalankan fitur tampil data, tambah data, ubah data, dan hapus data.

Langkah-langkah tersebut sesuai dengan prosedur kerja Acara 10 pada BKPM.
□filecite□turn6file0□

4.1 Struktur Project

```
Bkpm_Acara_10/
├── app/
│   ├── Controllers/
│   │   ├── AuthController.php
│   │   ├── BaseController.php
│   │   └── MahasiswaController.php
│   ├── Models/
│   │   ├── BaseModel.php
│   │   └── Mahasiswa.php
│   ├── Repositories/
│   │   └── MahasiswaRepository.php
│   ├── Database.php
│   ├── helpers.php
│   └── Views/
├── config/
│   └── database.php
├── database/
│   └── bkpm_acara_7 (1).sql
└── public/
    ├── .htaccess
    └── index.php
```

4.2 Implementasi BaseController

File: app/Controllers/BaseController.php

```php
<?php

// Induk semua Controller (Inheritance).
// Method yang dipakai bersama (view & redirect) ditulis sekali di sini,
// lalu diwarisi oleh controller lain lewat "extends BaseController".
abstract class BaseController
{
    // Tampilkan file view. Isi $data menjadi variabel di view,
    // contoh: ['daftar' => $x] -> di view bisa langsung memakai $daftar
    protected function view(string $view, array $data = []): void
    {
        extract($data);
        require __DIR__ . '/../Views/' . $view . '.php';
    }

    // Pindah halaman, pesan (opsional) disimpan di session
```

```
    protected function redirect(string $path, string $message = '', string
$type = 'success'): void
    {
        if ($message !== '') {
            $_SESSION['alert'] = ['type' => $type, 'message' => $message];
        }
        header('Location: ' . url($path));
        exit;
    }
}
```

4.3 Implementasi BaseModel

File: app/Models/BaseModel.php

```
<?php

require_once __DIR__ . '/../Database.php';

// Induk semua class yang butuh akses database (Inheritance).
// Koneksi PDO diambil dari objek Database yang disuntikkan lewat constructor,
// sehingga class turunan cukup memakai $this->pdo.
abstract class BaseModel
{
    protected PDO $pdo;

    public function __construct(Database $db)
    {
        $this->pdo = $db->getConnection();
    }
}
```

4.4 Implementasi MahasiswaRepository

File: app/Repositories/MahasiswaRepository.php

```
<?php

require_once __DIR__ . '/../Models/BaseModel.php';
require_once __DIR__ . '/../Models/Mahasiswa.php';

// Repository Pattern: seluruh query SQL mahasiswa ada di class ini.
// Mewarisi BaseModel, jadi $this->pdo sudah tersedia dari class induk
// (Database disuntikkan lewat constructor BaseModel).
class MahasiswaRepository extends BaseModel
{
```

```php
    // Semua mahasiswa + nama prodi (JOIN). Jika $keyword diisi, cari nama atau
NIM dengan LIKE
    public function all(string $keyword = ''): array
    {
        $sql = "SELECT m.*, p.nama AS prodi_nama
                FROM mahasiswa m
                JOIN prodi p ON m.prodi_id = p.id";
        $params = [];

        if ($keyword !== '') {
            $sql .= " WHERE m.nama LIKE :nama OR m.nim LIKE :nim";
            $params['nama'] = '%' . $keyword . '%';
            $params['nim']  = '%' . $keyword . '%';
        }

        $sql .= " ORDER BY m.nim";

        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($params);

        $hasil = [];
        foreach ($stmt->fetchAll() as $row) {
            $hasil[] = Mahasiswa::dariBaris($row);
        }
        return $hasil;
    }

    public function find(int $id): ?Mahasiswa
    {
        $stmt = $this->pdo->prepare("SELECT * FROM mahasiswa WHERE id = :id");
        $stmt->execute(['id' => $id]);
        $row = $stmt->fetch();

        return $row ? Mahasiswa::dariBaris($row) : null;
    }

    public function create(Mahasiswa $m): void
    {
        $stmt = $this->pdo->prepare(
            "INSERT INTO mahasiswa (nim, nama, email, prodi_id, angkatan)
             VALUES (:nim, :nama, :email, :prodi_id, :angkatan)"
        );
        $stmt->execute([
            'nim'      => $m->getNim(),
            'nama'     => $m->getNama(),
```

```
            'email'    => $m->getEmail(),
            'prodi_id' => $m->getProdiId(),
            'angkatan' => $m->getAngkatan(),
        ]);
    }

    public function update(Mahasiswa $m): void
    {
        $stmt = $this->pdo->prepare(
            "UPDATE mahasiswa
             SET nim = :nim, nama = :nama, email = :email, prodi_id =
:prodi_id, angkatan = :angkatan
             WHERE id = :id"
        );
        $stmt->execute([
            'nim'      => $m->getNim(),
            'nama'     => $m->getNama(),
            'email'    => $m->getEmail(),
            'prodi_id' => $m->getProdiId(),
            'angkatan' => $m->getAngkatan(),
            'id'       => $m->getId(),
        ]);
    }

    public function delete(int $id): void
    {
        $stmt = $this->pdo->prepare("DELETE FROM mahasiswa WHERE id = :id");
        $stmt->execute(['id' => $id]);
    }

    // Daftar prodi untuk pilihan di form
    public function daftarProdi(): array
    {
        return $this->pdo->query("SELECT id, kode, nama FROM prodi ORDER BY
kode")->fetchAll();
    }
}
```

4.5 Implementasi MahasiswaController

File: app/Controllers/MahasiswaController.php

```
<?php

require_once __DIR__ . '/BaseController.php';
```

```php
require_once __DIR__ . '/../Repositories/MahasiswaRepository.php';

// Mewarisi BaseController: view() dan redirect() dipakai dari class induk
class MahasiswaController extends BaseController
{
    private MahasiswaRepository $repo;

    // Constructor injection: Repository diberikan dari luar (lihat
public/index.php)
    public function __construct(MahasiswaRepository $repo)
    {
        $this->repo = $repo;
    }

    public function index(): void
    {
        $keyword = trim($_GET['q'] ?? '');
        $daftar  = $this->repo->all($keyword);
        $this->view('mahasiswa/index', ['daftar' => $daftar, 'keyword' =>
$keyword]);
    }

    public function create(): void
    {
        $this->tampilForm(new Mahasiswa(), url('/mahasiswa/store'));
    }

    public function store(): void
    {
        [$m, $errors] = $this->ambilInput();

        if (!$errors) {
            try {
                $this->repo->create($m);
                $this->redirect('/mahasiswa', 'Data mahasiswa berhasil
ditambahkan.');
            } catch (PDOException $e) {
                $errors = $this->errorDatabase($e);
            }
        }

        $this->tampilForm($m, url('/mahasiswa/store'), $errors, $_POST);
    }

    public function edit(): void
```

```
    {
        $m = $this->repo->find((int) ($_GET['id'] ?? 0));
        if ($m === null) {
            $this->redirect('/mahasiswa', 'Data mahasiswa tidak ditemukan.',
'danger');
        }

        $this->tampilForm($m, url('/mahasiswa/update'));
    }

    public function update(): void
    {
        $id = (int) ($_POST['id'] ?? 0);
        if ($this->repo->find($id) === null) {
            $this->redirect('/mahasiswa', 'Data mahasiswa tidak ditemukan.',
'danger');
        }

        [$m, $errors] = $this->ambilInput($id);

        if (!$errors) {
            try {
                $this->repo->update($m);
                $this->redirect('/mahasiswa', 'Data mahasiswa berhasil
diperbarui.');
            } catch (PDOException $e) {
                $errors = $this->errorDatabase($e);
            }
        }

        $this->tampilForm($m, url('/mahasiswa/update'), $errors, $_POST);
    }

    public function destroy(): void
    {
        $this->repo->delete((int) ($_POST['id'] ?? 0));
        $this->redirect('/mahasiswa', 'Data mahasiswa berhasil dihapus.');
    }

    // Isi objek Mahasiswa lewat setter. Setter yang gagal validasi
menghasilkan pesan error.
    private function ambilInput(int $id = 0): array
    {
        $m = new Mahasiswa();
        $m->setId($id);
```

```php
        $errors = [];

        $input = [
            'nim'      => ['setNim',      $_POST['nim'] ?? ''],
            'nama'     => ['setNama',     $_POST['nama'] ?? ''],
            'email'    => ['setEmail',    $_POST['email'] ?? ''],
            'prodi_id' => ['setProdiId',  (int) ($_POST['prodi_id'] ?? 0)],
            'angkatan' => ['setAngkatan', (int) ($_POST['angkatan'] ?? 0)],
        ];

        foreach ($input as $field => [$setter, $nilai]) {
            try {
                $m->$setter($nilai);
            } catch (InvalidArgumentException $e) {
                $errors[$field] = $e->getMessage();
            }
        }

        return [$m, $errors];
    }

    // NIM sudah dipakai (kolom UNIQUE) -> error 23000
    private function errorDatabase(PDOException $e): array
    {
        if ($e->getCode() === '23000') {
            return ['nim' => 'NIM sudah terdaftar.'];
        }
        throw $e;
    }

    private function tampilForm(Mahasiswa $m, string $action, array $errors =
[], array $old = []): void
    {
        $this->view('mahasiswa/form', [
            'm'         => $m,
            'action'    => $action,
            'errors'    => $errors,
            'old'       => $old,
            'prodiList' => $this->repo->daftarProdi(),
        ]);
    }
}
```

```
    }
```

4.6 Implementasi Class Mahasiswa

File: app/Models/Mahasiswa.php

```php
<?php

class Mahasiswa
{
    // Semua atribut private: hanya bisa diakses lewat getter dan setter
    private int $id = 0;
    private string $nim = '';
    private string $nama = '';
    private string $email = '';
    private int $prodiId = 0;
    private int $angkatan;
    private string $prodiNama = '';   // hanya untuk tampilan (hasil JOIN)

    public function __construct()
    {
        $this->angkatan = (int) date('Y');
    }

    // Buat objek dari satu baris data database (data dari database dianggap
sudah valid)
    public static function dariBaris(array $row): self
    {
        $m = new self();
        $m->id        = (int) $row['id'];
        $m->nim       = $row['nim'];
        $m->nama      = $row['nama'];
        $m->email     = $row['email'];
        $m->prodiId   = (int) $row['prodi_id'];
        $m->angkatan  = (int) $row['angkatan'];
        $m->prodiNama = $row['prodi_nama'] ?? '';
        return $m;
    }

    // ---------- Getter ----------
    public function getId(): int          { return $this->id; }
    public function getNim(): string      { return $this->nim; }
    public function getNama(): string     { return $this->nama; }
    public function getEmail(): string    { return $this->email; }
    public function getProdiId(): int     { return $this->prodiId; }
    public function getAngkatan(): int    { return $this->angkatan; }
    public function getProdiNama(): string { return $this->prodiNama; }
```

```php
    // ---------- Setter (dengan validasi sederhana) ----------
    public function setId(int $id): void
    {
        $this->id = $id;
    }

    public function setNim(string $nim): void
    {
        $nim = trim($nim);

        if ($nim === '') {
            throw new InvalidArgumentException('NIM wajib diisi.');
        }
        if (!ctype_digit($nim)) {
            throw new InvalidArgumentException('NIM harus berupa angka.');
        }
        if (strlen($nim) > 20) {
            throw new InvalidArgumentException('NIM maksimal 20 digit.');
        }

        $this->nim = $nim;
    }

    public function setNama(string $nama): void
    {
        $nama = trim($nama);

        if ($nama === '') {
            throw new InvalidArgumentException('Nama tidak boleh kosong.');
        }
        if (strlen($nama) > 100) {
            throw new InvalidArgumentException('Nama maksimal 100 karakter.');
        }

        $this->nama = $nama;
    }

    public function setEmail(string $email): void
    {
        $email = trim($email);

        if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException('Format email tidak valid.');
        }
```

```
        $this->email = $email;
    }

    public function setProdiId(int $prodiId): void
    {
        if ($prodiId <= 0) {
            throw new InvalidArgumentException('Pilih program studi.');
        }

        $this->prodiId = $prodiId;
    }

    public function setAngkatan(int $angkatan): void
    {
        if ($angkatan < 1990 || $angkatan > (int) date('Y') + 1) {
            throw new InvalidArgumentException('Angkatan tidak valid.');
        }

        $this->angkatan = $angkatan;
    }
}
```

4.7 Perakitan Object pada Entry Point

File: public/index.php

```
<?php

class Mahasiswa
{
    // Semua atribut private: hanya bisa diakses lewat getter dan setter
    private int $id = 0;
    private string $nim = '';
    private string $nama = '';
    private string $email = '';
    private int $prodiId = 0;
    private int $angkatan;
    private string $prodiNama = '';   // hanya untuk tampilan (hasil JOIN)

    public function __construct()
    {
        $this->angkatan = (int) date('Y');
    }
```

```php
    // Buat objek dari satu baris data database (data dari database dianggap
sudah valid)
    public static function dariBaris(array $row): self
    {
        $m = new self();
        $m->id        = (int) $row['id'];
        $m->nim       = $row['nim'];
        $m->nama      = $row['nama'];
        $m->email     = $row['email'];
        $m->prodiId   = (int) $row['prodi_id'];
        $m->angkatan  = (int) $row['angkatan'];
        $m->prodiNama = $row['prodi_nama'] ?? '';
        return $m;
    }

    // ---------- Getter ----------
    public function getId(): int          { return $this->id; }
    public function getNim(): string      { return $this->nim; }
    public function getNama(): string     { return $this->nama; }
    public function getEmail(): string    { return $this->email; }
    public function getProdiId(): int     { return $this->prodiId; }
    public function getAngkatan(): int    { return $this->angkatan; }
    public function getProdiNama(): string { return $this->prodiNama; }

    // ---------- Setter (dengan validasi sederhana) ----------
    public function setId(int $id): void
    {
        $this->id = $id;
    }

    public function setNim(string $nim): void
    {
        $nim = trim($nim);

        if ($nim === '') {
            throw new InvalidArgumentException('NIM wajib diisi.');
        }
        if (!ctype_digit($nim)) {
            throw new InvalidArgumentException('NIM harus berupa angka.');
        }
        if (strlen($nim) > 20) {
            throw new InvalidArgumentException('NIM maksimal 20 digit.');
        }

        $this->nim = $nim;
```

```
    }

    public function setNama(string $nama): void
    {
        $nama = trim($nama);

        if ($nama === '') {
            throw new InvalidArgumentException('Nama tidak boleh kosong.');
        }
        if (strlen($nama) > 100) {
            throw new InvalidArgumentException('Nama maksimal 100 karakter.');
        }

        $this->nama = $nama;
    }

    public function setEmail(string $email): void
    {
        $email = trim($email);

        if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
            throw new InvalidArgumentException('Format email tidak valid.');
        }

        $this->email = $email;
    }

    public function setProdiId(int $prodiId): void
    {
        if ($prodiId <= 0) {
            throw new InvalidArgumentException('Pilih program studi.');
        }

        $this->prodiId = $prodiId;
    }

    public function setAngkatan(int $angkatan): void
    {
        if ($angkatan < 1990 || $angkatan > (int) date('Y') + 1) {
            throw new InvalidArgumentException('Angkatan tidak valid.');
        }

        $this->angkatan = $angkatan;
    }
}
```

```
}
```

BAB V HASIL DAN PEMBAHASAN

5.1 Hasil Implementasi

Implementasi pada ZIP menunjukkan bahwa struktur Acara 10 telah memisahkan tanggung jawab antarclass. BaseController menyediakan fungsi umum controller, BaseModel menyediakan PDO untuk class turunan, dan MahasiswaRepository menangani query database. MahasiswaController kemudian menggunakan Repository untuk operasi data.

5.2 Penerapan Inheritance

MahasiswaController menggunakan deklarasi extends BaseController. Dengan demikian, MahasiswaController dapat menggunakan method view() dan redirect() dari BaseController tanpa menulis ulang method tersebut.

```
class MahasiswaController extends BaseController
{
    private MahasiswaRepository $repo;

    public function __construct(MahasiswaRepository $repo)
    {
        $this->repo = $repo;
    }
}
```

5.3 Penerapan Repository Pattern

MahasiswaRepository menjadi tempat operasi database mahasiswa. Method all(), find(), create(), update(), dan delete() menggunakan PDO dan prepared statement. Controller hanya memanggil method Repository tersebut.

5.4 Penerapan BaseModel

MahasiswaRepository mewarisi BaseModel. BaseModel menerima objek Database pada constructor dan mengambil koneksi PDO melalui getConnection(). Dengan begitu Repository dapat menggunakan $this->pdo untuk menjalankan query.

5.5 Alur Program

- Browser mengirim request ke public/index.php.
- public/index.php membangun objek Database.
- Objek Database diberikan kepada MahasiswaRepository melalui constructor BaseModel.
- MahasiswaRepository diberikan kepada MahasiswaController melalui constructor.
- MahasiswaController menjalankan method sesuai request.
- MahasiswaController meminta data atau menjalankan operasi melalui MahasiswaRepository.

- Repository menjalankan query database menggunakan PDO.
- Hasil dikembalikan ke Controller dan ditampilkan melalui View.

5.6 Pengujian Fitur

No	Fitur	Pengujian	Hasil yang Diharapkan
1	Tampil data	Membuka halaman /mahasiswa setelah login.	Daftar mahasiswa tampil.
2	Tambah data	Membuka form tambah, mengisi data valid, lalu Simpan.	Data baru tersimpan dan muncul pada daftar.
3	Ubah data	Memilih Edit, mengubah data, lalu menyimpan.	Data mahasiswa diperbarui.
4	Hapus data	Memilih Hapus dan mengonfirmasi.	Data mahasiswa terhapus.

Pengujian di atas mengikuti target pengujian pada prosedur Acara 10, yaitu memastikan fitur tampil, tambah, ubah, dan hapus data berjalan setelah penerapan Inheritance dan Repository Pattern. □filecite□turn6file0□

5.7 Pembahasan

Setelah refactoring, Controller tidak lagi menangani query SQL secara langsung. Fungsi umum controller dipindahkan ke BaseController, sedangkan kebutuhan akses PDO dasar ditempatkan pada BaseModel. Pemisahan tersebut membuat kode lebih terstruktur dan mengurangi pengulangan kode.

BAB VI KESIMPULAN

Pada Acara 10 telah diterapkan konsep Inheritance dan Repository Pattern pada aplikasi Manajemen Data Mahasiswa. BaseController digunakan sebagai class induk controller, sedangkan BaseModel menjadi dasar bagi class yang membutuhkan koneksi PDO. MahasiswaRepository menangani seluruh akses database sehingga MahasiswaController tidak perlu menuliskan query SQL secara langsung.

Dengan pembagian tanggung jawab tersebut, struktur aplikasi menjadi lebih rapi, mudah dipelihara, dan lebih mudah dikembangkan. Pengujian diarahkan pada fitur tampil, tambah, ubah, dan hapus data sesuai prosedur kerja Acara 10.